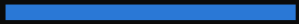


Searching the settings of a slow simulation

A plan for automatic tuning, and where to take it next



What the code does, and what it costs

A fusion reactor holds plasma at over a hundred million degrees. The exhaust leaving that plasma has to be cooled on its way out, or it destroys the metal wall it lands on. This code predicts whether a given reactor design manages that.

Simulating a tenth of a second of plasma, on ten cores:

~1 hour

a smaller production case

the cheapest thing worth running

10s of hours

a large case at full resolution

the ones the science needs

tens

cases in a single study

multiply by the row above

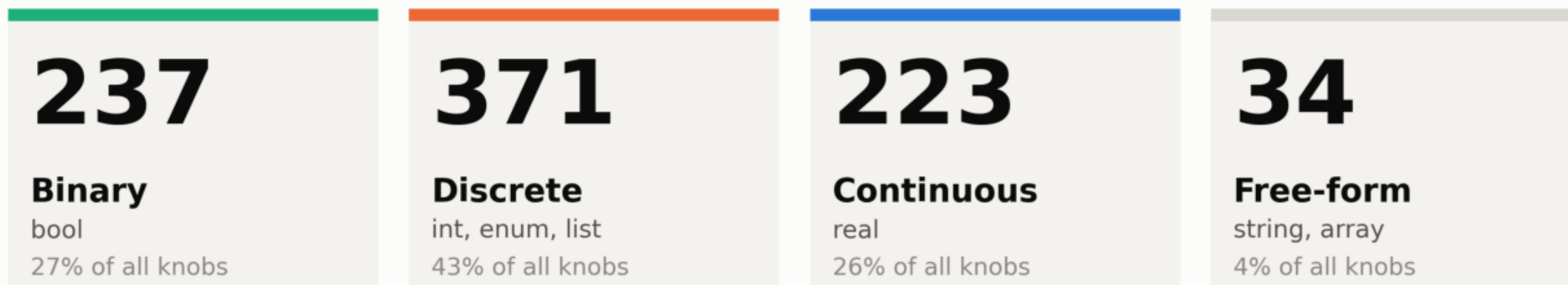
The speed of the code decides which questions can be asked at all.

The solver parameter space

Every knob the nonlinear solver stack exposes, counted by the type of value it takes.

865 tuneable knobs

831 of them take a value worth searching. 57 come from the calling code's own input file; the rest are the solver library's.



One knob per row of the inventory. Counts are exact; nothing on this figure is estimated. 785 of the 865 are confirmed present in the library build actually linked: 728 solver-library names found in it plus 57 from the calling code. The other 80 are mostly preconditioners this build does not compile, so absence here is weaker evidence than presence.

Most of the space does not exist yet

A knob is only real once its parent option is set to the right value. Block width is the number of knobs.

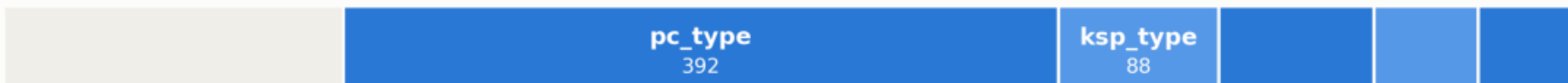
1. the whole space



2. does it exist by default?



3. which parent option switches it on



4. which value of that parent — 58 blocks, and only one value per parent can be chosen



Narrow blocks in row 3, in order: ■ snes_type 85 ■ solver:type 57 ■ pc_factor_mat_solver_type 56 ■ snes_linesearch_type 1

pc_type alone gates 392 of the 865 knobs — 45% of the space. It accepts 52 values, 26 of which bring knobs of their own, and they are mutually exclusive: choose one and you keep at most 133 of those knobs and delete the rest.

The parent is read from the depends column. A knob with no depends entry counts as always present.

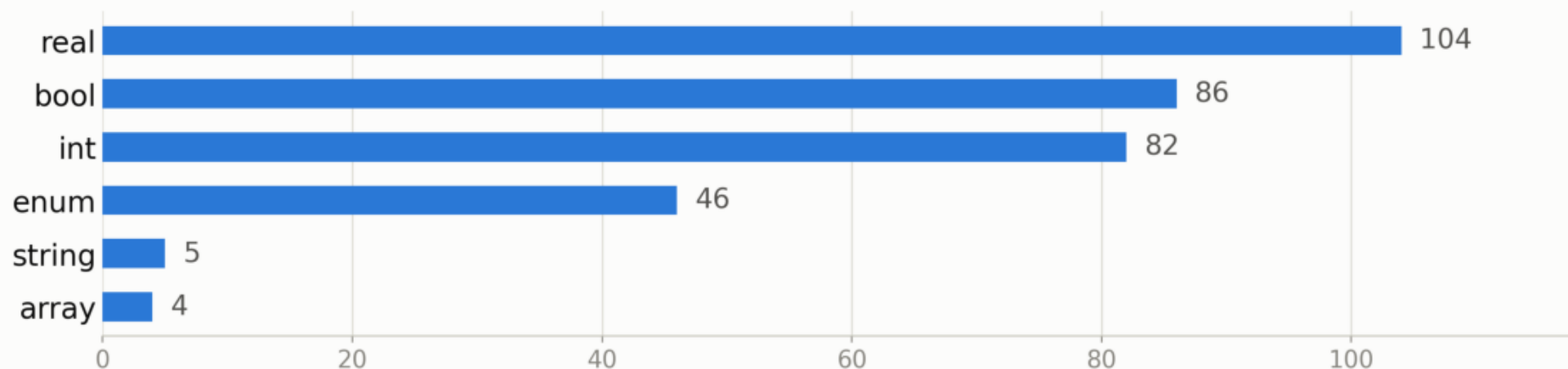
One point in that space is already this big

The configuration being run today: `solver:type = snes · snes_type = newtonls · ksp_type = fgmres · pc_type = lu · pc_factor_mat_solver_type = mumps · snes_linesearch_type = basic`

327

tuning knobs are live in this one configuration, before anyone changes a single type choice — 38% of the 865 in the whole space.

What kind of value those 327 knobs take



Change `pc_type` from `lu` to `hypr` and 19 of these knobs vanish while 133 different ones appear. The space does not just move — it changes shape.

Live means every clause of the knob's depends column is satisfied by the configuration named above.

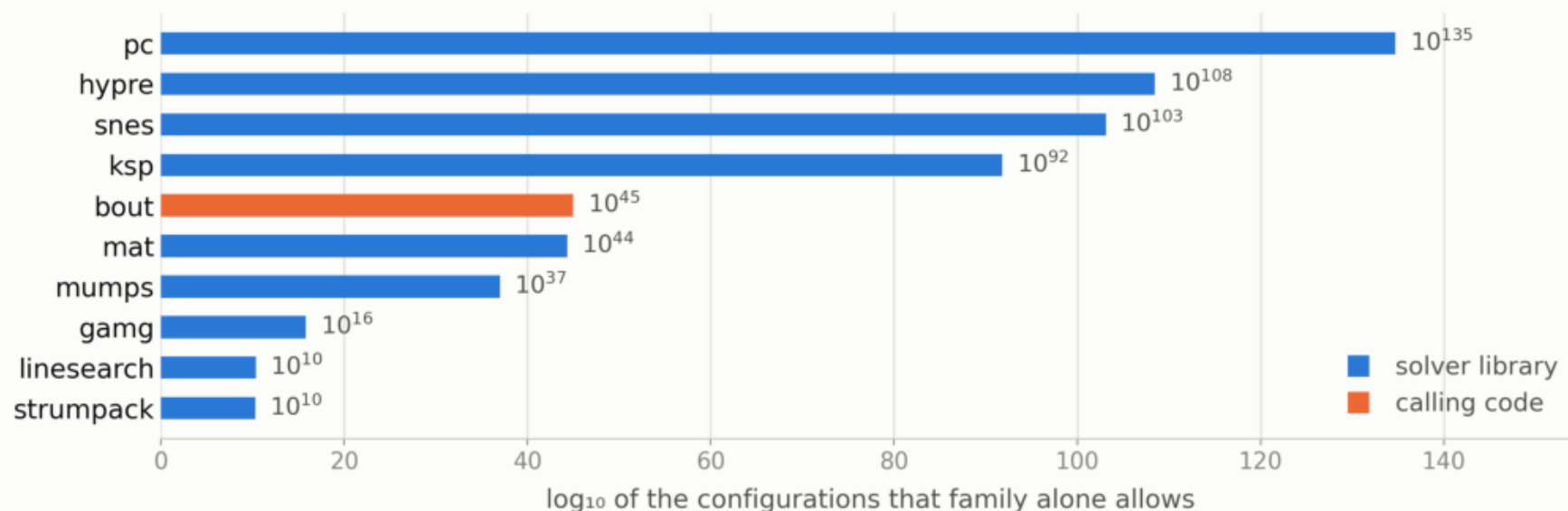
How big that makes the whole space

Give every knob a deliberately coarse number of values and multiply.

10⁶⁰¹

distinct configurations, under the assumptions at the foot of this figure.

For scale: about 10^{80} atoms in the observable universe, and 10^6 evaluations in a very generous compute budget.



Assumed values per knob: bool = 2, enum = 4, list = 4, int = 8, real = 10. Strings and arrays count as 1: they are not searched. Every knob is treated as independently settable, which the conditional structure says it is not, so this is the size of the box that contains the tree. 94 of the 120 enums state their choice count in the source and use it; the other 26 are registered at run time, the source does not say, and they are assumed to offer 4. The 223 real-valued knobs are each sampled at 10 points; a real knob is an interval, not a list, so the true space is uncountable and this figure is a floor.

Why this is not ordinary tuning

hours

One evaluation is expensive

so the number of trials will be small, and a method that needs thousands of them is out.

noisy

The same settings run twice do not agree

the measurement has spread, so a difference of a few per cent means nothing.

wrong

Fast can mean wrong

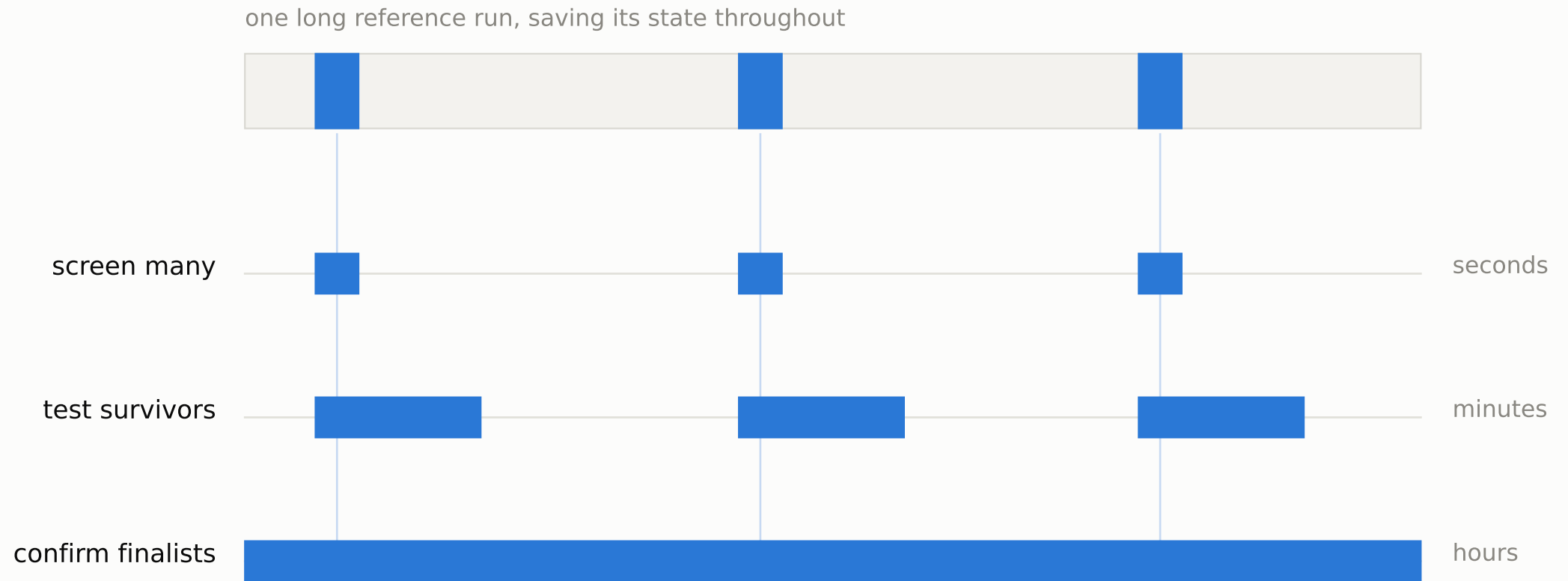
a setting can be quick because it is cutting corners. Speed alone cannot be the score.

failed

A failure is a result

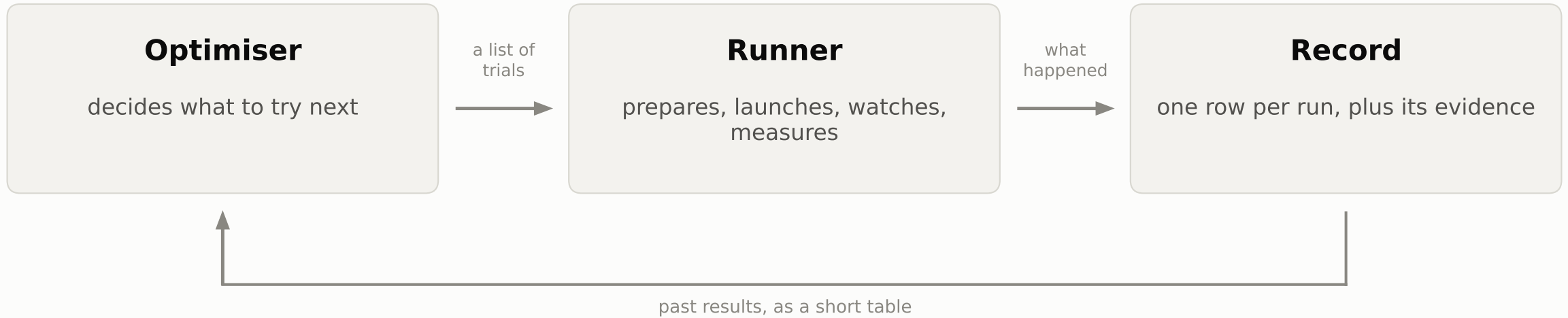
some settings make a run die or never finish. That is information, not a missing number.

Testing on a slice, not a whole run



A slice restarts from a saved state and runs a short way. Several positions, because the simulation is hard in different ways at different stages. Cheap slices screen many candidates; only the survivors pay for the longer ones.

The system



- The runner has no judgement in it, so it runs unattended, anywhere.
- The optimiser can be an algorithm, a language model, or a person writing a list. The runner cannot tell the difference, so it can be swapped without touching anything else.
- Speed and correctness are recorded side by side, never combined, so nothing can trade accuracy for time.

What exists, and what does not

Built and working

- Measurement. A finished run turns automatically into one row of numbers and a bundle of evidence.
- The record. One row per run, added to and never rewritten.
- Reference runs for three cases, with their saved states.
- The tool that cuts a slice and sets it up as a run.
- The loop itself. A four-trial campaign has been generated, run, measured and recorded with nobody watching.

Not built

- Any optimiser at all. Every trial so far was a list written by hand.
- Any evidence that a slice predicts a full run.
- A description of more than a fifth of the settings that are live.

What to work on together

- 1 Does a slice predict a full run, and how should that be checked before it is trusted?
- 2 What search method suits 865 mixed settings, most of which exist only when another setting has a particular value?
- 3 Where, if anywhere, does a language model belong in the loop?
- 4 How should a run that fails, or never finishes, be scored?